e-Prime - Advances in Electrical Engineering, Electronics and Energy 8 (2024) 100546

Contents lists available at ScienceDirect

e-Prime - Advances in Electrical Engineering, Electronics and Energy

journal homepage: www.elsevier.com/locate/prime

Design of floating point multiplier using approximate hybrid Radix-4/ Radix-8 booth encoder for image analysis

Pranose J. Edavoor [a], Aswini K. Samantaray [b,*], Amol D. Rahulkar [c]

[a] C-DAC, Bengaluru, India
[b] Department of Electronics and Communications Engineering, Vignan's Foundation for Science, Technology and Research, Guntur, India
[c] Department of Electrical and Electronics Engineering, National Institute of Technology Goa, India

ARTICLE INFO

Keywords:
Floating point multiplier
Booth encoder
Radix-4/Radix-8
Image smoothing

ABSTRACT

This paper presents a novel power efficient Hybrid floating point multiplier (HFPM) with approximate hybrid radix-4/radix-8 booth encoder. This approach efficiently increases the speed of operation and optimises hardware. Compared to existing HFPMs, the proposed HFPM utilises a novel hybrid radix-4/radix-8 booth encoder. The hybrid architecture reduces the computations by an average of k/3 for a k-bit 2's complement representation by reducing the number of partial products. In addition, to achieve an area-efficient and power-efficient design, approximate computational units are added to the radix-8 booth encoder in the hybrid architecture with a slight reduction in accuracy. The efficiency of the approximate architecture is measured in scales of Error Rate (ER) and Normalized Error Distance (NED). The proposed design is implemented on Kintex 7 FPGA and realized on 45nm ASIC platforms. The experimental study shows that the proposed multiplier has utilised hardware (831 slice LUT, 828 LUT as logic) and consumed 6110 μm^2 of area and 83 μW of power which is less as compared to existing floating point multipliers. The experimental results in terms of average Structural Similarity Index Measure (SSIM) is 0.98 which is better than the existing floating point multipliers are compared.

1. Introduction

Arithmetic operations such as Multiplication, division, addition, subtraction, cubing, squaring, etc. are performed by Arithmetic Logic Units (ALUs), which are fundamental components of processors. ALU of some processors are divided into Logic Unit (LU) and Arithmetic Unit (AU). Floating point operations are frequently used by a floating point unit on a separate chip known as a numeric co-processor in personal computers (PCs). In ALUs, multiplication is the most basic and commonly utilised operation. It allows one number to be scaled by another number. Floating point multiplication is the most common arithmetic operation for real-value arithmetic units with varied precision for multimedia processing, scientific computations, numerical processing, neural networks, and other applications [1]. The dynamic range of numbers that can be expressed in the floating point format is enormous since it can represent extremely small and very large values. All the operations in wavelet and neural network based multimedia applications utilise floating point arithmetic and hence, there is a need to develop floating point computational units (multipliers) with shorter path delay, smaller hardware utilisation and less power consumption.

The Booth Algorithm [2] is an approach that can be used to implement a floating point multiplier. The amount of partial products that need to be calculated can be decreased by using this algorithm. Modified Booth technique can decrease the amount of calculations in exchange for design complexity by applying this technique at higher radii. Radix-8 is thought to provide the optimum balance between speed and complexity for the development of digital circuits since it can reduce the number of partial products that must be calculated in half and only requires the addition of a bit-shift function as an extra operation. The literature survey regarding various floating point multiplier including booth multiplier is presented in the following section.

2. Review on existing floating point multipliers

The most widely used standard for Floating Point (FP) multiplication was defined by IEEE [3]. The multiplication operation using floating point numbers involves sign estimation, exponent estimation, mantissa estimation, product estimation, rounding off and normalisation to generate the final product. Various techniques have been introduced to increase the efficiency of FP multiplication operation. Kuang et al. [4]

* Corresponding author.
E-mail address: aswini.566@gmail.com (A.K. Samantaray).

https://doi.org/10.1016/j.prime.2024.100546
Received 11 October 2023; Received in revised form 25 February 2024; Accepted 14 April 2024
Available online 18 April 2024
2772-6711/© 2024 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

proposed a FP multiplier using latency scheme for low power applications. However, it requires more number of clock cycles to process the data. Dinechin et al. [5] proposed a large sized FP multiplier with fewer DSP blocks and implemented it on FPGA. This multiplier has increased power consumption due to the use of DSP blocks. As the technology is scaling down, Single Precision (SP) operations are replaced by double and higher precision computations [6–11]. Akkas and Schulte [8] designed a dual Double Precision (DP) and Quadruple Precision (QP) FP multiplier. Lang and Bruguera [9] proposed a DP FP adder multiplier with reduced latency using Leading Zero Anticipator (LZA) [10]. Tan et al. [6] proposed a FP multiplier to perform two parallel SP multiplication in each cycle and the latency was found to be of two clock cycles. Huang et al. [12] designed a quadruple precision FP fused Adder Multiplier (AM) based on Single Instruction Multiple Data (SIMD) method. However, it consumes more area and power.

In all the above higher precision multipliers, the basic computational unit is a single precision multiplier that is used iteratively or in parallel to realise higher precision operations. Mantissa multiplication operation needs to be optimised to reduce area, delay and power. The multiplication operation (after segregating the multiplicand and multiplier from the floating point representation) can be carried out using different algorithms like Dadda multiplier [13], Wallace tree multiplier [14], Booth multiplier [14], etc. Booth [2] suggested a standard for signed binary multipliers in first generation computers, which reduced the number of partial products by effectively utilising a uniform shift method. The main advantage of Booth multiplier is that, multiplier decoding can be done in either direction and sign correction is not required after the operation. In this, the multiplier is analysed in groups of bits (usually 2, 3, 4 etc) to compute the multiple of the multiplicand which is added to obtain the final product. Based on the grouping (2 bits, 3 bits and 4 bits), Booth multipliers can be classified as radix-2, radix-4 and radix-8 respectively. As the bits in a group increases, the number of partial products reduces, but the system becomes slow. In order to strike a balance between the speed of operation, area and power, a hybrid architecture for Booth encoding based multiplier has been proposed in [15]. This architecture was further modified to increase the efficiency for multimedia applications in portable devices. In the modified hybrid multiplier, a Hybrid Radix-4/Radix-8 Truncation (HRT) scheme was proposed to achieve the power efficiency with a tolerable reduction in Peak Signal to Noise Ratio (PSNR) in Graphics Processing Unit (GPU) applications [16]. Further, these architectures can be optimised for power and area by introducing approximate computational units. Approximation has been a popular technique to achieve high speed of operation in error resilient applications.

Tong et al. [17] have proposed a floating point multiplier design that optimises the power by minimising and varying the bit-width representation. A Mitchell's algorithm based accuracy configurable power efficient FP multiplier was proposed by Zhang et al. [18] and applies double bit truncation schemes to floating point multipliers. Zhang et al. [19] have proposed an imprecise floating point arithmetic unit for GPUs to achieve power saving. It is observed in [19] that, the introduction of approximation in FP multiplication reduces area, delay and power to a great extent without affecting the performance of FP multipliers. Also, it is found that FPGA based implementation is best suited for the high speed computing architectures because of its low power requirements, modular design, parallelism and pipelining [20].

Applications based on residue number system (RNS) modulation addition and multiplication can achieve lower power consumption, shorter latency, and smaller hardware area [27]. It is possible to execute several modulus operations concurrently when utilizing the multi-modulus architecture. Using a three-code interpretation, the conventional modified Booth-encoded multiplier is also known as a radix-4 Boothencoded multiplier [28,29]. The multiplier encodes the data in groups of three bits by adding a 0 after the least significant bit and before the most significant bit. This requires only n/2 for partial products with n bits. The radix-8 [30–32] architecture can increase the modulation multiplication encoding selection from a three-code to a four-code interpretation, which decreases the use of partial products from n/2 to n/3 + 1. This is in contrast to the previous radix-4 research. However, there will be an extra multiplication by 3 (× 3) and multiplication by 4 (× 4) operations to be handled for the radix-8 multiplier. Using the same carry-free principle in the RNS, the multiplication by 4 (× 4) operation can return twice, shifting left by two bits. Nevertheless, the hard multiple generator (HMG) requires multiplication by 1 (× 1) and multiplication by 2 (× 2) of the initial multiplicand in order to obtain multiplication by 3 (× 3). Delay, power consumption, and hardware costs all rise as a result. It is crucial to simplify the HMG for a triple operation as a result.

It is observed from the literature that the existing approaches exhibit high latency and need a substantial amount of hardware resources due to the complex nature of floating-point arithmetic. The complicated arithmetic units in these systems still require a significant amount of chip surface. In high-performance computing applications in particular, the current designs need more power. Approximate computing techniques allow to compromise accuracy for improved efficiency in terms of power, area, and performance. Thus, it is necessary to design an architecture with reduced area and power without affecting the latency. In order to achieve a power and area efficient floating point multiplier, this chapter explains a novel floating point multiplier architecture.

The contributions of this work are summarised below.

- Proposed a novel Hybrid Floating Point Multiplier (HFPM) unit to compute one single precision multiplication operation.
- A new scheme based on Booth encoding is proposed to increase the computational speed by reducing the number of partial product and to reduce power of proposed HFPM by using approximate computational units. An approximate hybrid radix-4/radix-8 Booth encoder is proposed.
- Introduced a novel architecture for approximate radix-8 Booth encoder that reduces area, delay and power of the hybrid Booth encoder by using less number of partial products.
- Implemented the proposed HFPM on Kintex 7 FPGA and on ASIC platform.
- Validated the proposed HFPM by employing in wavelet based image smoothing application.

The rest of the chapter is structured as follows. Architecture of the proposed approximate hybrid radix-4/radix-8 Booth encoder based floating point multiplier (HFPM) is presented in Section 3. Section 4 discusses the experimental results followed by wavelet based image smoothing application using proposed HFPM. The concluding remarks is presented in Section 5.

3. Proposed hybrid floating point multiplier architecture

This section gives a brief idea of floating point multiplication operation along with detailed description of the proposed hybrid floating point multiplier architecture.

3.1. Review of floating point multiplication

Let A and B represent two floating point numbers represented by equations (1) and (2) respectively.

$$A = (-1)^{s_A} m_A \times (2)^{Exp_A} \tag{1}$$

$$B = (-1)^{s_B} m_B \times (2)^{Exp_B} \tag{2}$$

The product of A and B, P can be represented as

$$P = (-1)^{s_P} m_P \times (2)^{Exp_P} \tag{3}$$

$$P = (-1)^{s_A \oplus s_B} m_A m_B \times (2)^{(Exp_A + Exp_B - bias)} \tag{4}$$

$$(-1)^{s_P} m_P \times (2)^{e_P} = (-1)^{s_A \oplus s_B} m_A m_B \times (2)^{(Exp_A + Exp_B - bias)} \quad (5)$$

Here, s_P, m_P, and e_P are the sign bit, mantissa and exponent of the product $(A \times B)$.

Upon estimation of the mantissa, exponent and sign, mantissa is appended with a '1' before the multiplication operation. The product estimated is normalized and the exponent is incremented by one if the number of digits in the decimal part exceeds two.

3.2. Proposed hybrid floating point multiplier architecture

The proposed HFPM is designed for single precision multiplication operation. The suggested architecture is shown in Fig. 1. This architecture can process (multiplication) two Single Precision Floating Point (SPFP) numbers at a time in a pipelined manner. The proposed architecture is divided into three phases and are as follows:

- **PHASE I**: Extraction and adjustment stage:- The mantissa, exponent and sign part are extracted in this stage. The extracted mantissa is prefixed with a 1 to get the actual value of the number represented using the standard floating point representation.
- **PHASE II**: Sign, exponent processing and mantissa multiplication stage:- Booth multiplier with hybrid architecture using radix-4 and radix-8 Booth encoders is proposed. This multiplier performs the mantissa multiplication operation. Apart from product estimation, sign estimation are also carried out.
- **PHASE III**: Post processing and normalization:- In this phase, rounding off and normalization followed by post processing are carried out.

The pipelined architecture is implemented to facilitate high speed multiplication operations, as it enables the processing of number pairs during each clock cycle. In addition, all the input exceptions are also considered in Phase I to generate the final product. Upon extraction of mantissa, exponent and sign, they are fed to corresponding processing units. The mantissa is processed to obtain the significand by the adding 1 as a leading bit. The approximate hybrid radix-4/radix-8 Booth encoded multipliers are utilized to perform multiplication operation followed by accumulation, rounding, normalization and post processing.

3.3. PHASE I: Extraction and adjustment stage

In Phase I, two FP input operands A and B represented using equation (6) and equation (7) are processed to extract sign, exponent and mantissa.

$$A = (-1)^{s_A} m_A \times (2)^{Exp_A} \quad (6)$$

$$B = (-1)^{s_B} m_B \times (2)^{Exp_B} \quad (7)$$

In this phase, sign (1 bit), exponent (8 bits) and mantissa (23 bits) are extracted using part select logic. It is known that, sign bit is the most significand bit of the inputs. Next, the 8-bit exponent of A and B follow the sign bit. Remaining 23-bit mantissas are extracted from the least significand bits of A and B. The mantissas are prefixed with a 1 to obtain the actual value of inputs to the mantissa multiplier. Next, the extracted signs (S_A, S_B), 8-bit exponents (Exp_A, Exp_B) and corrected 24-bit mantissas $(1, m_A, 1, m_B)$ are passed to Phase II through pipelined registers.

3.4. PHASE II : Sign, exponent processing and mantissa multiplication stage

The operations in Phase II are categorized as:

1. Sign estimation.
2. Exponent estimation.
3. Proposed approximate hybrid radix-4/radix-8 Booth encoder for mantissa multiplier.

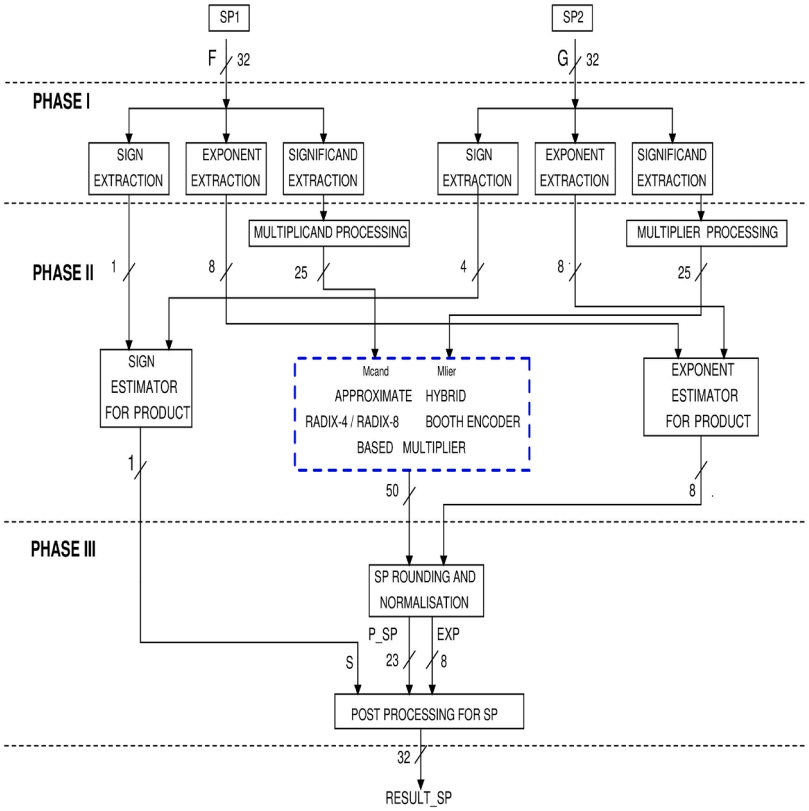


Fig. 1. Proposed architecture of HFPM.

These operations are explained as follows:

1. Sign estimation

In this operation, sign estimation is performed using an XOR gate represented by $\oplus$ and is expressed using equation (8).

$$S_P = S_A \oplus S_B \tag{8}$$

where,S_A, S_B and S_P are the sign bits of inputs A, B and the final product respectively.

2. Exponent estimation

Exponent processing for the final product is given by equation (9).

$$Exp_P = Exp_A + Exp_B - bias \tag{9}$$

where, Exp_A Exp_B and Exp_P are the exponents of inputs A, B and the final product respectively. This operation is realized using one 8-bit adder.

It is observed that, sign and exponent estimators do not affect the latency (critical path), because propagation delay of sign and exponent estimators is very less compared to propagation delay of mantissa multiplier. Hence, optimizations are required only in mantissa multiplier units.

Proposed approximate hybrid Radix-4/Radix-8 booth encoder for mantissa multiplier

A 25 × 25 approximate hybrid radix-4/radix-8 Booth encoder based multiplier is implemented in the proposed design. The mantissas processed (24-bit) in Phase I are fed to a multiplier in multiplicand processing unit. In this, the input mantissas (m_A, m_B) are processed into 25-bit inputs. Here, zeros are prefixed to significant bits of the corrected mantissas to ensure the inputs to the proposed Booth encoder based multiplier are positive 25-bit numbers.

This work uses a low power, high speed radix-4/radix-8 Booth encoder which integrates radix-4 and radix-8 Booth encoders to achieve low power and high speed multiplication operation. The properties and encoding schemes for radix-4 and radix-8 Booth encoders are shown in Table 1, Table 2 and Table 3.

It is observed from Table 1 and Table 2 that, the number of partial products in an N-bit multiplier is reduced to N/2 and can be implemented using shifter logic in radix-4 encoding schemes. This in turn aids in high speed operation.

2

Similarly, it is found from Table 1 and Table 3 that, radix-8 is a power efficient encoding scheme with lesser number of partial products N/3 compared to radix-4. However, radix-8 is slower than radix-4 due to ±3Y estimation as shown in Table 3 where one adder and one shifter is required to produce ±3Y. Thus, carry propagation delay in this adder adversely affects the speed of operation in radix-8 encoding.

In order to solve this issue, this work proposes an approximate radix-8 Booth encoder that replaces ±3Y terms with ±4Y terms. This modification in radix-8 encoding logic replaces the use of adders by simple shifting logic that are computationally less intensive and has better speed of operation.

Proposed approximate Radix-8 booth encoder

In this work, a hybrid architecture for Booth multiplier is proposed which utilizes both radix-4 and radix-8 Booth encoders. To further reduce area, delay and power, the radix-8 Booth encoder is optimized by applying principles of approximation. A brief overview of proposed approximate radix-8 Booth encoder and the hybrid Booth multiplier are presented.

In this, +3Y encoding is approximated to +4Y encoding which can be implemented using shift operation. Similarly, -3Y encoding is approximated to -4Y in order to replace adders by shifters in the encoding logic. Therefore, a deviation of ±1 is introduced and equal deviation of error in positive and negative directions (bounded error) is obtained. The proposed approximate encoding scheme is presented in Table 4 and is compared with exact radix-8 encoding scheme. The approximated encoding scheme presented in Table 4 is reduced using K-map and its gate level implementation is shown in Fig. 3. The encoding scheme for the proposed single partial product bit is expressed using equation (10).

$$\begin{aligned} PP_{jk} &= \big[b_{j-2}(\overline{a_{k+2}}a_{k+1}(a_k + a_{k-1}) + a_{k+2}\overline{a_{k+1}}(\overline{a_k} + \overline{a_{k-1}})) \\ &\quad + b_{j-1}(\overline{a_{k+1}}a_k a_{k-1} + a_{k+1}\overline{a_k a_{k-1}}) + b_j((\overline{a_{k+2} \oplus a_{k+1}})(a_k \oplus a_{k-1}))\big] \oplus a_{k+2} \end{aligned} \tag{10}$$

Proposed hybrid architecture using approximate Radix-4/Radix-8 booth encoder for 25 × 25 multiplier

The proposed multiplier operation unfolds as follows. The inputs to the proposed hybrid multiplier architectures are 25 bits multiplier and multiplicand (corrected mantissas) of m_A and m_B. Out of these 25 bits, lower significand twelve bits are encoded using proposed approximate radix-8 Booth encoders and the remaining higher significand 13 bits are encoded using exact radix-4 Booth encoders. This hybrid encoding logic for 25 × 25 multiplier is illustrated in Fig. 4. The approximate radix-8 encoding scheme is used in least significant bits so that the approximations do not reduce the accuracy of the multiplier considerably.

Consider a conventional 3 × 3 multiplication shown in Fig. 5 to

Table 1
Properties of Radix-4 and Radix-8 Booth encoders

Booth Encoder	No. of partial products	Utility	Speed	Power consumption
Radix-8	N/3	Limited	Slow	Low
Radix-4	N/2	Wide	Fast	High

Table 2
Encoding for Radix-4 Booth encoder.

Inputs			Partial Products (P_k)	PP_{jk}	Booth select	
a_{k+1}	a_k	a_{k-1}			$2P_k$	P_k
0	0	0	0	0	-	0
0	0	1	Y	b_j	-	1
0	1	0	Y	b_j	-	1
0	1	1	2Y	b_{j-1}	1	0
1	0	0	-2Y	$\overline{b_{j-1}}$	1	0
1	0	1	-Y	$\overline{-b_j}$	0	1
1	1	0	-Y	$\overline{-b_j}$	0	1
1	1	1	0	0	0	0

Table 3
Encoding for Radix-8 Booth encoder.

Inputs				Partial Product (P_k)	Booth select		
a_{k+2}	a_{k+1}	a_k	a_{k-1}		$4P_k$	$2P_k$	P_k
0	0	0	0	0	0	0	0
0	0	0	1	Y	0	0	1
0	0	1	0	Y	0	0	1
0	0	1	1	2Y	0	1	0
0	1	0	0	2Y	0	1	0
0	1	0	1	3Y	0	1	1
0	1	1	0	3Y	0	1	1
0	1	1	1	4Y	1	0	0
1	0	0	0	-4Y	1	0	0
1	0	0	1	-3Y	0	1	1
1	0	1	0	-3Y	0	1	1
1	0	1	1	-2Y	0	1	0
1	1	0	0	-2Y	0	1	0
1	1	0	1	-Y	0	0	1
1	1	1	0	-Y	0	0	1
1	1	1	1	0	0	0	0

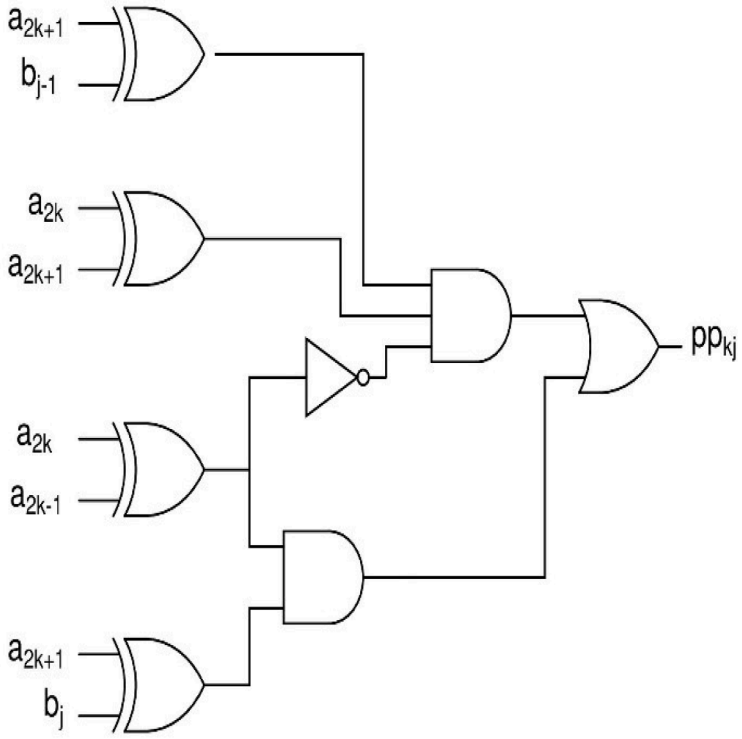


Fig. 2. Partial product generation for exact Radix-4 Booth encoder

Table 4
Encoding for proposed approximate Radix-8 Booth encoder.

Inputs				Exact Partial Product (P_k)	PP_{jk}	Proposed Partial Products($P_k^{'}$)	ED	$PP_{jk}^{'}$
a_{k+2}	a_{k+1}	a_k	a_{k-1}					
0	0	0	0	0	0	0	0	0
0	0	0	1	Y	b_j	Y	0	b_j
0	0	1	0	Y	b_j	Y	0	b_j
0	0	1	1	2Y	b_{j-1}	2Y	0	b_{j-1}
0	1	0	0	2Y	b_{j-1}	2Y	0	b_{j-1}
0	1	0	1	3Y	s_i	4Y	1	b_{j-2}
0	1	1	0	3Y	s_i	4Y	1	b_{j-2}
0	1	1	1	4Y	b_{j-2}	4Y	0	b_{j-2}
1	0	0	0	-4Y	$\overline{b_{j-2}}$	-4Y	0	$\overline{b_{j-2}}$
1	0	0	1	-3Y	s_i	-4Y	1	$\overline{b_{j-2}}$
1	0	1	0	-3Y	s_i	-4Y	1	$\overline{b_{j-2}}$
1	0	1	1	-2Y	$\overline{b_{j-1}}$	-2Y	0	$\overline{b_{j-1}}$
1	1	0	0	-2Y	$\overline{b_{j-1}}$	-2Y	0	$\overline{b_{j-1}}$
1	1	0	1	-Y	$\overline{b_j}$	-Y	0	$\overline{b_j}$
1	1	1	0	-Y	$\overline{b_j}$	-Y	0	$\overline{b_j}$
1	1	1	1	0	0	-Y	0	0

understand the levels and columns in a multiplication operation. It has three levels with 3 bits and five columns of partial product bits. Thus, n levels with n-bits and 2n-1 columns for partial product bits are required for conventional n × n multiplication. This work attempts to reduce the number of levels of partial product bits. The proposed hybrid Booth multiplier architecture implemented for 25 × 25 multiplication operation has eleven levels of 25-bit partial products with forty nine columns. Thus, the proposed architecture reduced the number of partial product levels by fourteen as opposed to twenty five levels required in conventional multiplier.

In the forty nine partial product columns, the most significant twenty five columns (from left side) are generated using exact radix-4 Booth encoding and the least significant twenty four columns are produced by the approximate radix-8 encoding. The incorporation of approximation in the floating point multiplier is limited to the least significant columns to ensure that the approximation does not adversely affect the accuracy of operation.

In addition, sign bit correction techniques are used in partial product accumulation to ensure proper product estimation in case of negative partial products. Finally, the last level of partial product is generated using a reduced encoding logic for exact radix-4. The output of multiplier in Phase II (50-bit) is part selected to 48-bit LSB value. This 48 bit product along with 1-bit output from sign estimator and 9-bit output (including carry bit) from exponent estimator is passed to Phase III through pipelined registers.

3.5. Phase III: Post processing

In this phase, post processing is carried out using rounding and normalization [2]. The 48-bit output from Phase II is utilized to generate Guard bit-Round bit-Sticky bit (GRS Bit) to determine the rounding off operation. The 48-bit product is divided into two 24-bit parts *upper_product* [22] and *lower_product* [21]. The *lower_product* determines the rounding operation of *upper_product* by generating GRS bit. Firstly, AND operation is performed on two bits from *lower_product* and *upper_product* [22]. Secondly, rest of the bits in *lower_product* are ORed. Both these results are ORed to generate the final GRS bit which is added to the *upper_product* for rounding operation. During rounding operation, if an over flow occurs, the *upper_product* and the exponent are adjusted to reflect the rounding off accurately. The modified *upper_product* is reduced to 23-bit mantissa of the final product by removing the most significand bit (hidden bit '1'). Finally, sign bit (1-bit) and the adjusted exponent (8-bit) and mantissa (23-bit) are combined to form the final IEEE 754 representation of the product. This approach ensures that maximum accuracy is retained by employing exact radix-4 booth encoders for generating mantissa (*upper_product*).

4. Experimental results

This section explains experimental results for the proposed hybrid

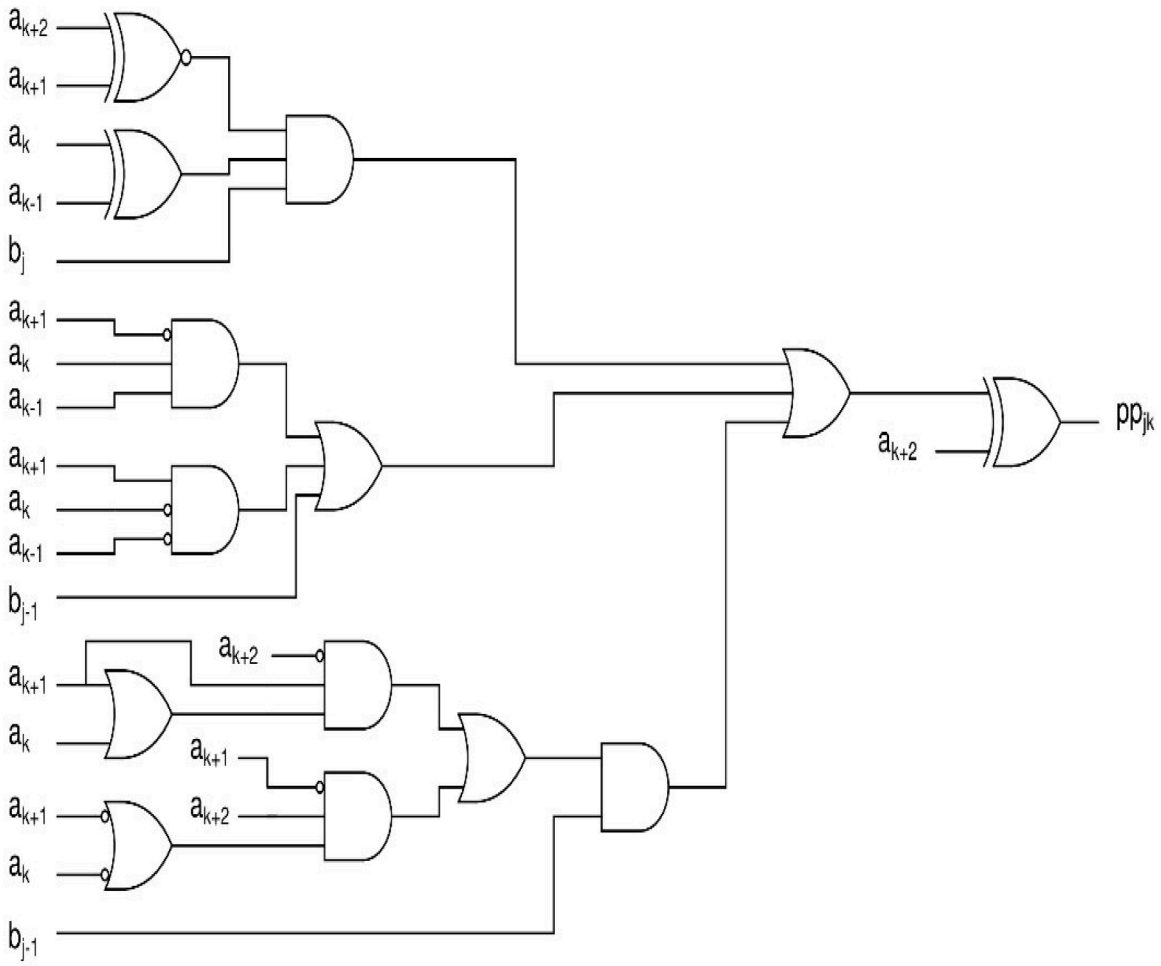


Fig. 3. Proposed Radix-8 Booth encoder.

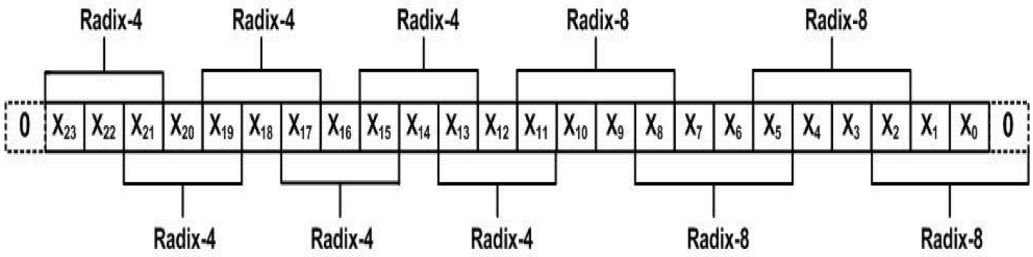


Fig. 4. Proposed hybrid encoding logic for mantissa multiplier.

floating point multiplier and a comparison with existing state-of-the-art floating point multiplier designs is also presented.

4.1. Efficiency measure of proposed approximate hybrid Radix-4/Radix-8 booth multiplier

The proposed approximate radix-4/radix-8 hybrid Booth multiplier along with existing multipliers are implemented using structural modelling in Verilog and verified on NC-Simulator provided by Cadence. The synthesis of these circuits are carried out using 45nm technology at room temperature with a supply voltage of 1V.

The comparison of area, delay power and Normalised Error Distance (NED) of the proposed with existing implemented multipliers are shown in Table 5. It is observed from Table 5 that, the proposed approximate radix-4/radix-8 hybrid Booth multiplier reports the best area, power and delay compared to all the existing designs. In order to measure the accuracy of the proposed approximate multiplier designs, an error metric - NED is also measured and compared. NED is defined as the mean error distance normalised with the maximum output of the exact design. From Table 5 it can be observed that, the proposed approximate radix-4/radix-8 hybrid Booth multiplier achieved the minimum NED compared to other multiplier designs.

The comparison of area, delay and power of the hybrid floating point multiplier along with the existing exact and approximate floating point multipliers [22–24] is listed in Table 6. It is observed from Table 6 that, the convential Radix-4 and radix-8 Booth multipliers are implemented with 8008 μm^2 and 9191 μm^2 area and 1262.36 μW and 1381 μW power respectively dye to their originality. Other existing techniques consume less area and power than the original multipliers. However, the proposed HFPM consume area and power of 6110 μm^2 and 948.31 μW

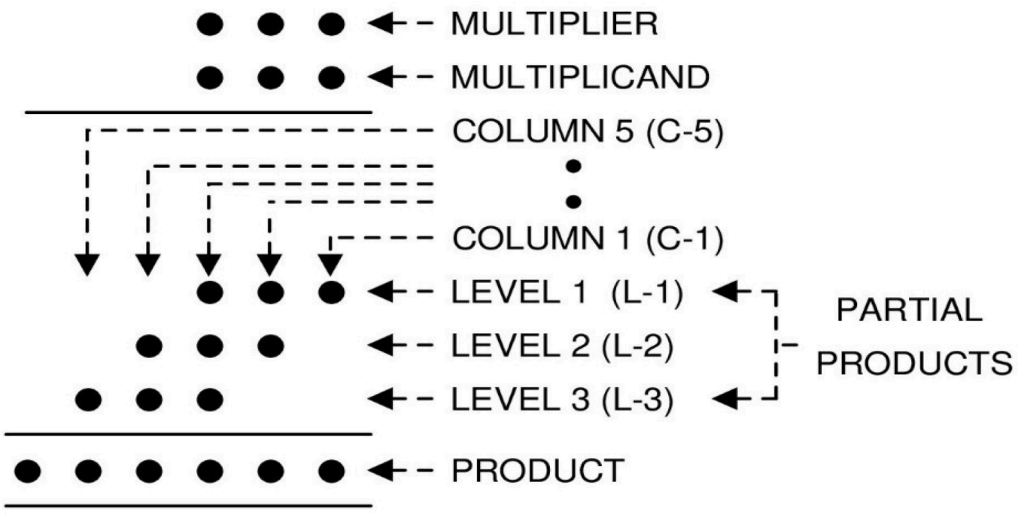


Fig. 5. Partial product generation for 3×3 multiplier.

Table 5
Comparison of area, delay and power for existing and proposed mantissa multipliers using booth encoders.

Design	Area (μm^2)	Power (μW)	Delay (ns)	NED($\times 10^{11}$)
Conventional Radix-8	6003	898.54	1.49	-
Conventional Radix-4	7197	937.2	1.53	-
Jiang et al. [22]	5414	858.8	1.20	4.38
Liu et al. [23]	5131	829.7	1.12	1.97
Leon et al. [24]	4482	714.2	0.98	9.91
Fu et al. [29]	4326	702.6	0.98	2.56
Kuo and Wu [28]	4122	686.2	0.94	1.92
Proposed	4076	677.9	0.93	1.86

Table 6
Comparison of area and power of proposed HFPM with conventional SP floating point multiplier and existing floating point multiplier unit.

Design	Area (μm^2)	Power (μW)
Conventional Radix-8	8008	1262.36
Conventional Radix-4	9191	1381.71
Jiang et al. [22]	7408	1198.39
Liu et al. [23]	7125	1168.45
Leon et al. [24]	6487	1028.57
Fu et al. [29]	6344	1005.16
Kuo and Wu [28]	6208	986.54
Proposed	6110	948.31

respectively. Hence, proposed HFPM reduces area and power significantly compared to existing floating point multipliers. This is achieved by using the approximate computational units in the proposed HFPM.

4.2. FPGA implementation results

The HFPM architecture is simulated, synthesised and verified with Kintex 7 FPGA board from Xilinx. Verilog HDL is used to design HFPM and Vivado 2016.4 is used for simulation.

The experimental study has been carried out to understand the improvement of the proposed design over well-known existing floating point multipliers with one million test cases. Table 7 shows the hardware utilisation of HFPM architecture with three approximate floating point multipliers [22–24,28,29] and two exact floating point multipliers in order to validate FPGA hardware utilisation of the proposed HFPM. It is observed from Table 7 that, the proposed HFPM utilised same number of LUTs as memory and flip-flops. At the same time, the proposed HFPM uses 831 slice LUTs and 828 LUT as logic which is the least compared to all existing designs. This is due to the proposed design uses less number of partial products.

Table 8 shows the power utilisation for HFPM on Kintex 7 FPGA. It is seen that the proposed architecture shows reduced power consumption when compared to all conventional architectures and approximate architectures. This is due the use of less number of slice LUT and LUT as logic in the proposed HFPM as shown in Table 7.

4.3. Use of HFPM in wavelet based image smoothing application

The convolution operation involves multiplication and addition

Table 8
Power Consumption for existing architecture and proposed HFPM architecture.

Design	Power (μW)
Conventional Radix-8	149
Conventional Radix-4	158
Jiang et al. [22]	98
Liu et al. [23]	124
Leon et al. [24]	107
Fu et al. [29]	96
Kuo and Wu [28]	86
Proposed	83

Table 7
Hardware utilisation of floating point multipliers and the proposed HFPM implemented on Kintex 7 FPGA platform.

Site type	Available	Conv. Radix-8	Conv Radix-4	Jiang et al. [22]	Liu et al. [23]	Leon et al. [24]	Fu et al. [29]	Kuo and Wu [28]	Proposed HFPM
Slice LUT	303600	920	965	902	944	911	885	849	831
LUT as logic	303600	917	962	899	941	908	882	846	828
LUT as memory	130800	3	3	3	3	3	3	3	3
Flip-Flops	607200	267	267	267	267	267	267	267	267

operations. It is found from the literature that, the CDF 9/7 wavelet FB consists of floating point filter coefficients. Thus, the proposed HFPM is tested in real-time wavelet based smoothing application. In this, multiplication operations are carried out using the proposed HFPM and addition is implemented using floating point adders. Also, existing floating point multipliers are used in image smoothing application to validate the efficacy of the proposed design. Image smoothing application is simulated in Matlab 2019.2. The experimental results of the proposed HFPM and the existing floating point multipliers are compared in terms of average SSIM [25].

6

Structural Similarity Index Measure (SSIM)- a perceptual metric is utilised to evaluate image quality decline by determining the similarity of two images (by extracting information such as correlation, contrast, and luminance).

$$SSIM(l.m) = [lum(l,m)^{\omega}][con(l,m)^{\eta}][str(l,m)^{\epsilon}] \tag{11}$$

where l, m represents two image signals that are non-negative. lum(l,m), con(l,m) and str(l,m) are the luminance comparison, contrast comparison and structure comparison functions respectively. ω, η and ϵ are positive parameters that can be varied to change the weightage of luminance, structure and contrast while computing SSIM. MSSIM estimates the average SSIM in local area. The expression for MSSIM is given as

$$MSSIM(l,m) = \frac{1}{N}\sum_{i=1}^{N} SSIM(l_i, m_i) \tag{12}$$

Table 9 shows a comparative study of MSSIM for image smoothing application using CDF 9/7 wavelet for twelve standard images (Pirate, Tiffany, Moon, Jet, Room, Cameraman, Lena, Elaine, Mandrill, Bridge, Lake and Pepper) [26]. Conventional radix-4 and radix-8 have SSIM 1 as both the designs use exact multipliers. The proposed HFPM based design is able to achieve MSSIM of 0.98 which is better as compared to the approximate multiplier designs. In order to achieve an area-efficient and power-efficient design, approximate computational units are added to proposed HFPM with a slight reduction in accuracy. Hence, the proposed floating point multiplier can be used in real time multimedia applications.

Table 9
Comparison of MSSIM for image smoothing application for existing architectures and proposed HFPM architecture.

Design	MSSIM
Conventional Radix-8	1
Conventional Radix-4	1
Jiang et al. [22]	0.94
Liu et al. [23]	0.95
Leon et al. [24]	0.92
Fu et al. [29]	0.95
Kuo and Wu [28]	0.97
Proposed	0.98

5. Conclusion

This chapter introduced floating point multiplier based on a novel architecture for approximate radix-8 Booth encoder and approximate hybrid radix-4/radix-8 Booth multiplier. The efficiency of the proposed approximate hybrid radix-4/radix-8 Booth encoder based floating point multiplier is compared with recent state-of-the-art multiplier designs based on delay, power, area and NED. It is found that the proposed design has the least area, delay, and power as compared to recent designs. It is seen that reduction in area, delay and power is achieved with the proposed approximate radix-8 Booth encoder due to reduced addition operations and hardware complexity. The analysis was carried out on ASIC platform with 45nm technology. The proposed multiplier architecture was implemented on Kintex 7, simulated and tested using Vivado 2016.4. The implemented multiplier is verified in image smoothing application using wavelets. According to the experimental study, the proposed HFPM uses hardware (831 slice LUT, 828 LUT as logic), uses 6110 μm^2 of area and 83 μW of power, which is less than that of existing floating point multipliers. The average Structural Similarity Index Measure (SSIM) for the experimental results is 0.98, which is better than the existing floating point multipliers.

CRediT authorship contribution statement

Pranose J. Edavoor: Conceptualization, Methodology. **Aswini K. Samantaray:** Formal analysis, Software, Validation. **Amol D. Rahulkar:** Supervision.



Fig. 6. Smoothing result Smoothing result of Lena image using various methods. (from left to right (top: Radix 8, Radix 4, Jiang et al [22]), Liu et al. [23] (bottom: Leon et al. [24], Fu et al. [29] Kuo and Wu [28], Proposed))

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

[1] M.K. Jaiswal, R.C. Cheung, Area-efficient architectures for double precision multiplier on FPGA, with run-time-reconfigurable dual single precision support, Microelectron. J. 44 (5) (2013) 421–430.
[2] A.D. Booth, A signed binary multiplication technique, Q. J. Mech. Appl. Math. 4 (2) (1951) 236–240.
[3] IEEE standard for floating-point arithmetic, IEEE Standard 754-2008 (2008) 1–70.
[4] S.R. Kuang, J.P. Wang, H.Y. Hong, Variable-latency floating-point multipliers for low-power applications, IEEE Trans. Very Large Scale Integrat. (VLSI) Syst. 18 (10) (2010) 1493–1497.
[5] F. de Dinechin, B. Pasca, Large multipliers with fewer DSP blocks. 2009 International Conference on Field Programmable Logic and Applications, 2009, pp. 250–255.
[6] D. Tan, C.E. Lemonds, M.J. Schulte, Low-power multiple-precision iterative floating-point multiplier with SIMD support, IEEE Trans. Comput. 58 (2) (2009) 175–187.
[7] N.T. Quach, N. Takagi, M.J. Flynn, Systematic IEEE rounding method for high-speed floating-point multipliers, IEEE Trans. Very Large Scale Integrat. (VLSI) Syst. 12 (5) (2004) 511–521.
[8] S. A. Akka, M.J. Schulte, A decimal floating point fused multiply-add unit with a novel decimal leading zero anticipator. ASAP 2011- 22nd IEEE International Conference on Application Specific Systems, Architectures and Processors, 2011, pp. 43–50.
[9] J.D. Bruguera, T. Lang, Floating-point fused multiply-add: reduced latency for floating-point addition. 17th IEEE Symposium on Computer Arithmetic (ARITH'05), 2005, pp. 42–51.
[10] E. Hokenek, R.K. Montoye, Leading-zero anticipator (LZA) in the IBM RISC system/6000 floating-point execution unit, IBM J. Res. Dev. 34 (1) (1990) 71–77.
[11] S. Banescu, F. de Dinechin, B. Pasca, R. Tudoran, Multipliers for floating-point double precision and beyond on FPGAs, ACM SIGARCH Comput. Architect. News 38 (4) (2011) 73–79.
[12] L. Huang, S. Ma, L. Shen, Z. Wang, N. Xiao, Low-cost binary128 floating-point FMA unit design with SIMD support, IEEE Trans. Comput. 61 (5) (2012) 745–751.
[13] L. Dadda, Some schemes for fast serial input multipliers. 1983 IEEE 6th Symposium on Computer Arithmetic (ARITH), 1983, pp. 52–59.
[14] C.S. Wallace, A suggestion for a fast multiplier, IEEE Trans. Electron. Comput. EC-13 (1) (1964) 14–17.
[15] G. Kim, S. Lee, J. Park, H. Yoo, A low-energy hybrid radix-4/-8 multiplier for portable multimedia applications. 2011 IEEE International Symposium of Circuits and Systems (ISCAS), 2011, pp. 1171–1174.
[16] S. Choi, G. Kim, H. Yoo, B. Nam, Hybrid radix-4/-8 truncated multiplier for mobile GPU applications, Electron. Lett. 50 (23) (2014) 1680–1682.
[17] J.Y.F. Tong, D. Nagle, R.A. Rutenbar, Reducing power by optimizing the necessary precision/range of floating-point arithmetic, IEEE Trans. Very Large Scale Integrat. (VLSI) Syst. 8 (3) (2000) 273–286.
[18] H. Zhang, W. Zhang, J. Lach, A low-power accuracy-configurable floating point multiplier. 2014 IEEE 32nd International Conference on Computer Design (ICCD), 2014, pp. 48–54.
[19] H. Zhang, M. Putic, J. Lach, Low power GPGPU computation with imprecise hardware. 2014 51st ACM/EDAC/IEEE Design Automation Conference (DAC), 2014, pp. 1–6.
[20] S. Taslimi, R. Faraji, A. Aghasi, H.R. Naji, Adaptive edge detection technique implemented on FPGA, Iranian J. Sci. Technol. Trans. Electric. Eng. 44 (2020) 1571–1582.
[21] C. Zhang, P. Li, G. Sun, Y. Guan, B. Xiao, J. Cong, Optimizing fpga-based accelerator design for deep convolutional neural networks. Proceedings of the 2015 ACM/SIGDA International Symposium on Field Programmable Gate Arrays, 2015, pp. 161–170.
[22] H. Jiang, J. Han, F. Qiao, F. Lombardi, Approximate radix-8 booth multipliers for low-power and high-performance operation, IEEE Trans. Comput. 65 (8) (2016) 2638–2644.
[23] W. Liu, L. Qian, C. Wang, H. Jiang, J. Han, F. Lombardi, Design of approximate radix-4 booth multipliers for error-tolerant computing, IEEE Trans. Comput. 66 (8) (2017) 1435–1441.
[24] V. Leon, G. Zervakis, D. Soudris, K. Pekmestzi, Approximate hybrid high radix encoding for energy-efficient inexact multipliers, IEEE Trans. Very Large Scale Integrat. (VLSI) Syst. 26 (3) (2018) 421–430.
[25] Z. Wang, A.C. Bovik, H.R. Sheikh, E.P. Simoncelli, Image quality assessment: from error visibility to structural similarity, IEEE Trans. Image Process. 13 (4) (2004) 600–612.
[26] Sipi.usc.edu. SIPI image database [online], 2016,. Available: http://sipi.usc.edu/database/.
[27] C. Efstathiou, I. Kouretas, P. Kitsos, On the modulo 2n + 1 addition and subtraction for weighted operands, Microprocess. Microsyst. 11 (2023) 2138–2164.
[28] C.-T. Kuo, Y.C. Wu, FPGA implementation of a novel multifunction modulo (2n ± 1) multiplier using radix-4 booth encoding scheme, Appl. Sci. 13 (2023) 10407.
[29] C. Fu, X. Zhu, K. Huang, Z. Gu, An 8-bit radix-4 non-volitile parallel multiplier, Electronics 10 (2021) 2358.
[30] R. Kumar, R.K. Jaiswal, R.A. Mishra, Perspective and opportunities of modulo 2n - 1 multipliers in residue number system: a review, J. Circuits Syst. Comput. 29 (2020) 2030008.
[31] N.K. Kabra, Z.M. Patel, Area and power efficient hard multiple generator for radix-8 modulo 2n - 1, Integr. VLSI J. 75 (2020) 102–113.
[32] N.K. Kabra, Z.M. Patel, A radix-8 modulo 2n multiplier using area and power-optimized, IET Comput. Digit. Tech. 15 (2021) 36–55.



Pranose J. Edavoor received the bachelor's degree in electronics and communication engineering from Cochin University of Science and Technology, Kerala, India, in 2012, and the master's degree in very large-scale integration (VLSI) from the National Institute of Technology Goa, India, in 2016, where he his Ph.D. degree with the Electrical and Electronics Department in 2021. His research interests include digital design, FPGA accelerators, ASIC design,wavelets, and deep neural networks.



Aswini K. Samantaray received the B.Tech. degree in electronics and communication engineering from the Gandhi Institute of Engineering and Technology, Gunupur, India, in 2008, and the M.Tech. degree from the C. V. Raman College of Engineering, Bhubaneswar, India, in 2012. He completed his Ph.D. degree with the Department of Electrical and Electronics Engineering, National Institute of Technology Goa (NIT Goa), India in 2022. His research interests include the design of wavelets and filter-banks, image processing, and FPGA accelerators.



Amol D. Rahulkar (Member, IEEE) was born in India, in 1978. He received the B.E. degree in instrumentation engineering from the Shri Guru Gobind Singhji (SGGS) Institute of Engineering and Technology, Nanded, India, in 2000, the M.Tech. degree from the Indian Institute of Technology (IIT) Kharagpur, India, in 2002, and the Ph.D. degree from the SGGS Institute of Engineering and Technology, Nanded, affiliated to Swami Ramanand Teerth Marathwada University Nanded, India, in 2013. He is currently working as an Associate Professor with the Department of Electrical and Electronics Engineering, National Institute of Technology Goa (NIT Goa), India. His current research interests include the design of wavelets and filter-banks, digital signal processing, image processing, biometrics, FPGA accelerators, and soft-computing.